

Universidade do Vale dos Sinos

UNISINOS

Nome: Leandra Fernanda Vieira Friedrich

**Cadeira: Arquitetura de Software Aplicada: Sistema de Controle de Despesas
Pessoais**

Sumário

01- Introdução_____	03
02- Descrição das Tecnologias_____	04
03- Descrição da Arquitetura Escolhida_____	06
04- Reflexão_____	07
05- Referências_____	08

01 - Introdução

O Sistema proposto trata-se de um **Controle de despesas pessoais**, desenvolvido para auxiliar um usuário individual no registro e acompanhamento de seus gastos diários. O objetivo é fornecer uma visão clara nas finanças, permitindo o controle efetivo por meio do registro de despesas e do cálculo automático do total gasto.

O usuário é uma pessoa física que deseja gerenciar suas finanças.

As funcionalidades principais :

- | - Cadastrar as despesas;
- | - Remover as Despesas;
- | - Visualizar o total de despesas com filtro por período

As regras de negócio são:

- | - Toda despesa deve possuir uma descrição e um valor;
- | - Uma despesa deve possuir uma data;
- | - O usuário pode cadastrar despesas;
- | - O usuário pode remover despesas;
- | - O sistema deve calcular o total de despesas.

02 - Descrição das Tecnologias

Para a implementação, seriam utilizadas as seguintes tecnologias:

- |**- Linguagem: Python, devido à sua simplicidade e vasto ecossistema para aplicações back-end. A justificativa é a rápida prototipação e clareza na implementação das regras de negócio.
- |**- Framework: Flask (microframework), por ser leve e adequado para uma aplicação de escopo definido, com baixa complexidade. A justificativa é o foco nas funcionalidades essenciais sem sobrecarga de configuração.
- |**- Banco de Dados: SQLite, por ser um banco relacional embutido, que não requer servidor dedicado, ideal para um sistema pessoal e de usuário único. A justificativa é a simplicidade de implantação e integração com Python.

03 - Descrição da Arquitetura Escolhida

A arquitetura selecionada é a MVC (Model-View-Controller). Suas principais características são a separação clara em três componentes: Model (gerencia dados e regras de negócio), View (interface com o usuário) e Controller (orquestra as requisições e a interação entre Model e View).

Aplicação no sistema de controle de despesas e localização das regras de negócio:

| - Model : (Regras a, b, e): Define a classe Despesa com os atributos (descrição, valor, data) e os métodos calcular_total(). A validação de que toda despesa possui descrição e valor (regra a) e data (regra b) é implementada aqui. O cálculo do total (regra e) também está no Model.

| - View: São as telas HTML (formulário de cadastro, lista de despesas, botão de remover).

| - Controller (Regras c, d): O Controller mapeia as rotas. Por exemplo, ao receber um POST /cadastrar, ele chama o Model para criar a despesa (regra c). Ao receber DELETE /remover/{id}, chama o Model para excluir (regra d).

Exemplo concreto – Cadastro de uma despesa:

Usuário preenche descrição "Lanche", valor 25.50 e data 07/06/2026 na View.

| - A View envia os dados via POST para o Controller (/cadastrar).

| - O Controller instancia um objeto Despesa (validando descrição, valor e data no Model).

| - O Model salva no SQLite e retorna sucesso ao Controller.

| - O Controller renderiza a View atualizada com a nova despesa.

Exemplo concreto – Cálculo do total de gastos:

Usuário acessa a página "Relatório" na View.

A requisição vai ao Controller (/total).

O Controller chama o método estático calcular_total() do Model.

O Model consulta todas as despesas no banco, soma os valores e retorna o total.

O Controller envia o total para a View exibir.

Elementos Visuais:

Descrição textual do Diagrama Estrutural (para auxiliar na criação):

|- Camada View (Front-end): Formulários HTML, listas de despesas.

|- Camada Controller: app.py com rotas /cadastrar, /remove, /total.

|- Camada Model: Classe Despesa (id, descricao, valor, data) e método calcular_total().

|- Camada Banco de Dados: SQLite (arquivo despesas.db).

Descrição textual do Diagrama de Fluxo:

1.->Usuário -> 2. View (Formulário) -> 3. Controller (POST /cadastrar) -> 4. Model (valida e salva) -> 5. Banco de Dados -> 6. Model (retorno) -> 7. Controller -> 8. View (lista atualizada) -> 9. Usuário.

04 - Reflexão

A maior dificuldade ao aplicar o MVC foi definir claramente o limite de responsabilidade do Controller, evitando que ele assumisse lógica de negócio (como validar se o valor é numérico). Garantir que o Model concentrasse todas as regras (incluindo o cálculo do total) exigiu disciplina, mas o resultado foi um sistema mais testável e de manutenção previsível.

05 - Referências

SOMMERVILLE, Ian. Engenharia de Software. 10. ed. São Paulo: Pearson Education do Brasil, 2018.

PRESSMAN, Roger S. Engenharia de Software: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.